

HTML Iframes

HTML Iframes

What is an Iframe?

- `<iframe>` embeds **another HTML document** within the current page.
- Common uses: **videos, maps, widgets, advertisements, external content.**

```
<iframe src="https://www.example.com" width="600" height="400"> </iframe>
```

Iframe Attributes

Attribute	Purpose	Example
<code>src</code>	URL of embedded page	<code><iframe src="https://youtube.com"> </iframe></code>
<code>width / height</code>	Dimensions of iframe	<code><iframe width="800" height="600"> </iframe></code>
<code>frameborder</code>	Border visibility (0 = none)	<code><iframe frameborder="0"></code>
<code>allowfullscreen</code>	Allow fullscreen mode	<code><iframe allowfullscreen></code>
<code>name</code>	Target for links/forms	<code><iframe name="iframe1"></iframe></code>
<code>sandbox</code>	Restrict iframe actions	<code><iframe sandbox="allow-scripts"> </iframe></code>
<code>referrerpolicy</code>	Control referrer info	<code><iframe referrerpolicy="no-referrer"> </iframe></code>

Feedback:

- Use attributes correctly for **functionality and security**.
 - Sandbox ensures **untrusted content cannot manipulate your page**.
-

Responsive Iframes

- Fixed-size iframes are **not mobile-friendly**.
- **CSS trick** for responsive embedding:

```
<div style="position:relative; padding-bottom:56.25%; height:0;
overflow:hidden;">
  <iframe src="https://www.youtube.com/embed/abc123"
    style="position:absolute; top:0; left:0; width:100%;
height:100%;"
    frameborder="0" allowfullscreen>
  </iframe>
</div>
```

Maintains **16:9 aspect ratio** and adapts to screen size.

Feedback:

- Makes embedded content **scalable and mobile-compatible**.
 - Essential for modern web applications with **videos, maps, or external widgets**.
-

Iframe Security

- **Risks:** clickjacking, XSS, phishing
- **Best practices:**
 - Always use **sandbox** to restrict actions

- Limit **allow** permissions (**allow-scripts**, **allow-same-origin**)
- Use HTTPS URLs
- Set **referrerpolicy**

```
<iframe src="https://example.com"
        sandbox="allow-scripts allow-same-origin"
        referrerpolicy="no-referrer">
</iframe>
```

Feedback:

- Proper security ensures **user trust and safety**.
 - Avoid embedding **untrusted content** without restrictions.
-

HTML **<head>** Tag

- Contains **metadata, resources, scripts, and styles**.
- Not visible on page but crucial for **SEO, responsiveness, and performance**.

```
<head>
  <title>My Website</title>
  <meta charset="UTF-8">
  <meta name="description" content="Advanced HTML tutorial">
  <meta name="keywords" content="HTML, Iframes, Forms, Security">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
  <script src="script.js"></script>
</head>
```

Feedback:

- Metadata improves **search ranking** and page **usability**.
 - Correct charset ensures **international text displays correctly**.
-

HTML Entities & URI Encoding

HTML Entities

What Are HTML Entities?

- HTML uses **special characters** that have **reserved meanings** (like `<`, `>`, `&`).
- Entities allow **displaying these characters safely** without breaking the HTML structure.
- Format: `&entity_name;` or `&#entity_number;`

Common HTML Entities

Character	Entity Name	Numeric Code	Example
<code><</code>	<code>&lt;</code>	<code>&#60;</code>	Displays <code><</code>
<code>></code>	<code>&gt;</code>	<code>&#62;</code>	Displays <code>></code>
<code>&</code>	<code>&amp;</code>	<code>&#38;</code>	Displays <code>&</code>
<code>"</code>	<code>&quot;</code>	<code>&#34;</code>	Displays <code>"</code>
<code>'</code>	<code>&apos;</code>	<code>&#39;</code>	Displays <code>'</code>
©	<code>&copy;</code>	<code>&#169;</code>	Displays ©
®	<code>&reg;</code>	<code>&#174;</code>	Displays ®
€	<code>&euro;</code>	<code>&#8364;</code>	Displays €
™	<code>&trade;</code>	<code>&#8482;</code>	Displays ™

Usage Example

```
<p>HTML stands for &lt;HyperText Markup Language&gt; &amp; is fun!</p>
<p>(c) 2025 MyWebsite. All rights reserved.</p>
```

Feedback:

- HTML entities **prevent code-breaking** when using reserved characters.
- Improves **browser rendering** and **security** (avoid accidental tag injection).
- Always use **entities for special symbols** in text or content.

URI Encoding

What is URI Encoding?

- Some characters are **unsafe** or **reserved** in URLs (space, &, ?, #).
- URI encoding converts them into **percent-encoded format** (%XX).
- Ensures URLs are **valid, safe, and interpretable by browsers**.

Reserved Characters

Character	Encoded Value	Example
Space	%20	Hello World → Hello%20World
#	%23	page#section → page%23section

&	%26	name=Tom&age=25 → name=Tom%26age=25
?	%3F	search?q=test → search%3Fq%3Dtest
/	%2F	folder/file → folder%2Ffile

Example: Encoding a URL

```
let url = "https://example.com/search?q=HTML & CSS";
let encodedURL = encodeURIComponent(url);
console.log(encodedURL);
// Output: https://example.com/search?q=HTML%20%26%20CSS

let component = "HTML & CSS";
let encodedComponent = encodeURIComponent(component);
console.log(encodedComponent);
// Output: HTML%20%26%20CSS
```

Feedback:

- `encodeURIComponent()` → Encodes **full URL**, keeping valid URI characters intact.
 - `encodeURIComponent()` → Encodes **component/value** completely for safe query strings.
 - Prevents **broken URLs and security vulnerabilities** (like XSS or injection attacks).
-

HTML Frames (Legacy)

- Frames were used in **earlier versions of HTML** to divide the **browser window into multiple sections**, each displaying a **different HTML document**.
- **Purpose:** Allowed content to **remain static** in one frame while other frames could be scrolled or navigated independently.
- **Status:** Deprecated in **HTML5** → replaced by **CSS layouts** and `<iframe>` for embedding content.

```
<frameset cols="30%,70%">
  <frame src="menu.html">
  <frame src="content.html">
</frameset>
```

Feedback:

- Avoid using frames; use **semantic HTML + CSS for layout**.
-

Charset

- Defines **text encoding** to display characters correctly.
- UTF-8 is recommended.

```
<meta charset="UTF-8">
```

Feedback:

- Essential for **multi-language support** and emojis.
 - Prevents **garbled characters** on web pages.
-

HTML Forms & Form Elements

What is a Form?

- Collects **user input** → sends to **server**.
- Used in **login, signup, surveys, feedback, payments**.

Form Syntax & Attributes

```
<form action="/submit" method="post" enctype="multipart/form-data"
autocomplete="off">
  <!-- Input elements go here -->
</form>
```

Attribute	Purpose
action	Server URL
method	GET/POST
enctype	File encoding
autocomplete	Enable/disable browser suggestions
target	Response location

Common Form Elements

Element	Purpose	Example
<input>	Single-line input	<input type="text" name="username">
<textarea>	Multi-line input	<textarea rows="4"></textarea>
<select>	Dropdown	<select><option>HTML</option></select>
<option>	Dropdown option	<option value="js">JavaScript</option>

<code><label></code>	Binds text	<code><label for="email">Email</label></code>
<code><button></code>	Submit/reset/custom	<code><button type="submit">Submit</button></code>
<code><fieldset></code> & <code><legend></code>	Group elements	<code><fieldset><legend>User Info</legend></fieldset></code>
<code><datalist></code>	Auto-suggestions	<code><input list="browsers"><datalist id="browsers"><option>Chrome</option></datalist></code>
<code><output></code>	Display results	<code><output id="result"></code>

Input Attributes

Attribute	Purpose	Example
<code>type</code>	Text, email, password, number	<code><input type="email"></code>
<code>name</code>	Server identifier	<code><input name="username"></code>
<code>placeholder</code>	Hint text	<code><input placeholder="Enter name"></code>
<code>required</code>	Mandatory	<code><input required></code>
<code>readonly</code>	Non-editable	<code><input readonly></code>
<code>disabled</code>	Field inactive	<code><input disabled></code>
<code>pattern</code>	Regex validation	<code><input pattern="[A-Za-z]{3,}"></code>
<code>maxlength</code>	Maximum characters	<code><input maxlength="20"></code>

Form Methods

Method	Description	Use Case
GET	Data in URL	Search/filter
POST	Data in body	Login, signup, payments

Real-Life Example

```
<form action="/register" method="post">
  <fieldset>
    <legend>Sign Up</legend>
    <label for="user">Username:</label>
    <input type="text" id="user" name="username" required>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <label>Gender:</label>
    <input type="radio" name="gender" value="M"> Male
    <input type="radio" name="gender" value="F"> Female

    <label for="course">Course:</label>
    <select id="course" name="course">
      <option value="html">HTML</option>
      <option value="css">CSS</option>
      <option value="js">JavaScript</option>
    </select>

    <button type="submit">Register</button>
  </fieldset>
</form>
```

Feedback:

- Proper **grouping, labeling, validation** improves **UX and accessibility**.
- Forms are **interactive and crucial for web apps**.
- Security: validate **server-side**, use HTTPS, prevent XSS.